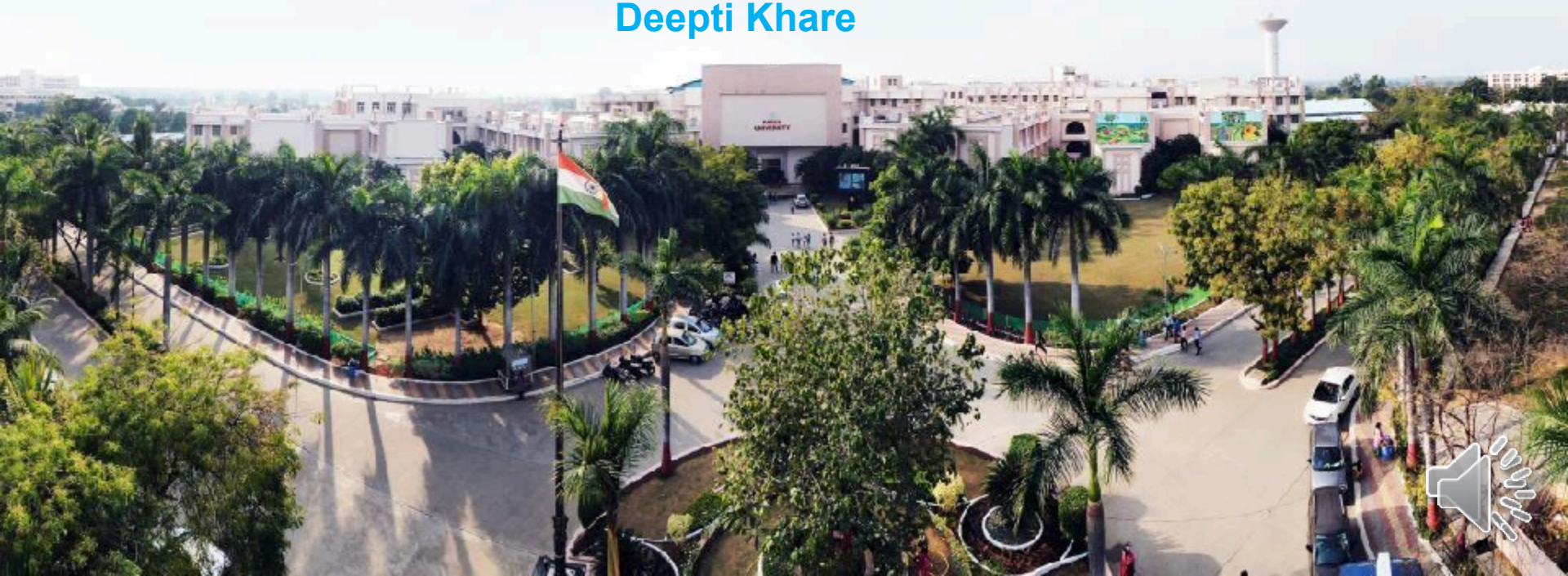
Parul® University

Full Stack Web Development

Created By :
Deepti Khare



Parul® University

DIGITAL LEARNING CONTENT

CHAPTER-7

Mongo DB

Parul® University

Introduction of MongoDB

MongoDB is a powerful, open-source **NoSQL database** that offers a **document-oriented data model**, providing a flexible alternative to traditional relational databases. Unlike SQL databases, MongoDB stores data in BSON format, which is similar to JSON, enabling efficient and scalable data storage and retrieval.

What is MongoDB?

MongoDB the most popular NoSQL database, is an **open-source document-oriented** database. The term 'NoSQL' means '**non-relational**'. This means that MongoDB isn't based on a **table-like relational database** structure but provides an altogether different mechanism for the **storage** and **retrieval** of data. This format of storage is called **BSON** (similar to **JSON** format). A simple MongoDB document Structure:

```
{
  title: 'Geeksforgeeks',
  by: 'Harshit Gupta',
  url: 'https://www.geeksforgeeks.org',
  type: 'NoSQL'
}
```

Parul® University

Introduction conti.

SQL databases store data in tabular format. This data is stored in a predefined data model which is not very much flexible for today's real-world highly growing applications.
Modern applications are more networked, social and interactive than ever. Applications are storing more and more data and are accessing it at higher rates.
Relational Database Management System(RDBMS) is not the correct choice when it comes to handling big data by the virtue of their design since they are not horizontally scalable. If the database runs on a single server, then it will reach a scaling limit.
NoSQL databases are more scalable and provide superior performance. MongoDB is such a NoSQL database that scales by adding more and more servers and increases productivity with its flexible document model.

Parul® University

(Intro conti.) Features of Mongo DB

Feature	What It Means	Benefits
Document-Oriented	Stores related data (e.g. components of a computer) in a single JSON/BSON document	Eliminates joins, simplifies data modeling, and speeds data access (mongodb.com, geeksforgeeks.org, boardinfinity.com)
Indexing	Supports primary, secondary, compound, geospatial, and text indexes	Enables fast, efficient query execution without collection scans
Horizontal Scalability	Uses sharding (partitioning by shard key) across multiple servers	Scales out seamlessly, supports large datasets, and balances load
Replication & High Availability	Replica sets maintain multiple copies with automatic failover	Ensures redundancy and uptime; seamless failover on server failure
Aggregation	Supports aggregation pipeline, map-reduce, and single-purpose methods	Enables grouped data processing, calculations (sum, avg, max, etc.)
High Performance	Combines embedding, indexing, and optimized storage engines	Delivers fast reads/writes and efficient storage
Ad-Hoc & Rich Queries	Supports real-time, flexible queries including regex, geospatial, and JavaScript	Ideal for analytics, searching, and dynamic query patterns
ACID Transactions	Offers multi-document transactions since v4.0	Ensures data integrity in complex operations
File Storage (GridFS)	Stores large files by chunking them into documents	Handles files larger than 16 MB with replication and balanced distribution

Parul® University

Intro conti.

Where do we use MongoDB?

MongoDB is preferred over RDBMS in the following scenarios:

Big Data: If we have huge amount of data to be stored in tables, think of MongoDB before RDBMS databases. MongoDB has built-in solution for partitioning and sharding our database.

Unstable Schema: **Adding a new column** in RDBMS is hard whereas MongoDB is **schema-less**. Adding a new field does not effect old documents and will be very easy.

Distributed data Since multiple copies of data are stored across different servers, recovery of data is instant and safe even if there is a hardware failure.

Language Support by MongoDB

MongoDB currently provides official driver support for all popular programming languages like C, C++, Rust, C#, Java, **Node.js, Perl, PHP, Python, Ruby, Scala, Go and Erlang.**

Parul® University

NoSQL DB

Introduction to NoSQL

NoSQL, or **"Not Only SQL,"** is a database management system (DBMS) designed to handle large volumes of unstructured and semi-structured data. Unlike traditional relational databases that use tables and pre-defined schemas, NoSQL databases provide **flexible data models** and support **horizontal scalability**, making them ideal for modern applications that require real-time data processing.

Why Use NoSQL?

Unlike relational databases, which uses Structured Query Language, **NoSQL databases don't have a universal query language**. Instead, each type of NoSQL database typically has its unique query language. Traditional relational databases follow **ACID (Atomicity, Consistency, Isolation, Durability)** principles, ensuring strong consistency and structured relationships between data.

NoSQL DB conti.

However, as applications evolved to handle **big data, real-time analytics, and distributed environments**, NoSQL emerged as a solution with:

Scalability – Can scale horizontally by adding more nodes instead of upgrading a single machine.

Flexibility – Supports unstructured or semi-structured data without a rigid schema.

High Performance – Optimized for fast read/write operations with large datasets.

Distributed Architecture – Designed for high availability and partition tolerance in distributed systems.

Parul® University

Four main types of NoSQL databases:

Type	Data Model	Strengths / Use Cases	Examples
Document	JSON/BSON documents	Flexible schema, nested data; ideal for CMS, profiles, catalogs	MongoDB, CouchDB, Cloudant (geeksforgeeks.org, noobtomaster.com, en.wikipedia.org, altexsoft.com)
Key-Value	Key → Value pairs	Ultra-fast lookups; caching, session storage, leaderboards	Redis, Memcached, Amazon DynamoDB
Column-Family	Column families	Scalable writes/analytics, time-series, IoT	Cassandra, HBase, Google Bigtable
Graph	Nodes & edges	Complex relationship queries: social graphs, fraud, recos	Neo4j, Amazon Neptune, ArangoDB

Parul® University

Advantages of NoSQL

Advantages

Flexible Schema – Supports dynamic, semi-structured data without migrations, enabling rapid development.

Horizontal Scalability – Easily shards data across servers to handle growing workloads smoothly.

High Performance – Optimized for high-speed reads/writes in distributed environments.

Ideal for Semi-Unstructured & Big Data – Handles JSON/XML/streaming data natively; great for IoT, analytics, social media.

High Availability & Replication – Built-in replication and failover mechanisms provide strong uptime.

Cost-Effective – Open-source options and use of commodity hardware reduce total cost of ownership.

Disadvantages of NoSQL

Disadvantages

Eventual Consistency – Sacrifices strict ACID consistency in favor of availability and performance, risking stale reads.

Limited Query Support – Lacks a standard query language like SQL; may require extra logic for complex queries.

Tooling & Ecosystem Gaps – Fewer mature tools and less comprehensive community support than for SQL databases.

Operational Complexity – Requires careful data modeling and management of sharding, replication, and backups.

Steep Learning Curve – Developers need to learn varied data models, APIs, and consistency paradigms.

Vendor/Model Lock-in – Each NoSQL type has unique APIs and behaviors, making migrations challenging.

RDBMS vs MongoDB

- RDBMS has a typical schema design that shows number of tables and the relationship between these tables whereas MongoDB is document-oriented. **There is no concept of schema or relationship.**
- Complex transactions are not supported in MongoDB because complex join operations are not available.
- MongoDB allows a highly flexible and scalable document structure. For example, one data document of a collection in MongoDB can have two fields whereas the other document in the same collection can have four.
- MongoDB is faster as compared to RDBMS due to efficient indexing and storage techniques.
- There are a few terms that are related in both databases. What's called Table in RDBMS is called a Collection in MongoDB. Similarly, a Row is called a Document and a Column is called a Field. MongoDB provides a default '_id' (if not provided explicitly) which is a 12-byte hexadecimal number that assures the uniqueness of every document. It is similar to the Primary key in RDBMS.

Installation of MongoDB

Step 1: Download MongoDB Community Server

Go to the MongoDB Download Center to download the MongoDB Community Server. Here, You can select any version, Windows, and package according to your requirement.

Step 2: Install MongoDB

When the download is complete open the **msi file** and click the **next button** in the startup screen:

Now accept the **End-User License Agreement** and click the next button:

Now select the **complete option** to install all the program features. Here, if you can want to install only selected program features and want to select the location of the installation, then use the **Custom option***:*

Step 3: Configure MongoDB Service

Select "**Run service as Network Service use**r" and copy the path of the data directory.

Click Next:

Click the **Install button** to start the MongoDB installation process:

After clicking on the install button installation of MongoDB begins:

Installation conti.

Step 4: Complete Installation

Now click the **Finish button** to complete the MongoDB installation process:

Step 5: Set Environment Variables

Now we go to the location where MongoDB installed in step 5 in your system and copy the **bin** path:

Now, to create an environment variable open system **properties >> Environment Variable >> System variable >> path >> Edit Environment variable**

paste the copied link to your environment system and **click Ok**:

Mongo DB – Data Types

MongoDB supports many datatypes. Some of them are −

String − This is the most commonly used datatype to store the data. String in MongoDB must be UTF-8 valid.

Integer − This type is used to store a numerical value. Integer can be 32 bit or 64 bit depending upon your server.

Boolean − This type is used to store a boolean (true/ false) value.

Double − This type is used to store floating point values.

Min/ Max keys − This type is used to compare a value against the lowest and highest BSON elements.

Arrays − This type is used to store arrays or list or multiple values into one key.

Timestamp − ctimestamp. This can be handy for recording when a document has been modified or added.

Object − This datatype is used for embedded documents.

Null − This type is used to store a Null value.

Symbol − This datatype is used identically to a string; however, it's generally reserved for languages that use a specific symbol type.

Parul® University

Mongo DB – Data Type conti.

Date − This datatype is used to store the current date or time in UNIX time format. You can specify your own date time by creating object of Date and passing day, month, year into it.

Object ID − This datatype is used to store the documents ID.

Binary data − This datatype is used to store binary data.

Code − This datatype is used to store JavaScript code into the document.

Regular expression − This datatype is used to store regular expression.

Parul® University

Mongo DB – Data Models

Data in MongoDB has a flexible schema documents in the same collection. They do not need to have the same set of fields or structure Common fields in a collections documents may hold different types of data.

Data Model Design

MongoDB provides two types of data models: **Embedded data model and Normalized data model.** Based on the requirement, you can use either of the models while preparing your document.

Data Model conti.

Embedded Data Model

In this model, you can have (embed) all the related data in a single document, it is also known as de-normalized data model.

For example, assume we are getting the details of employees in three different documents namely, Personal_details, Contact and, Address, you can embed all the three documents in a single one as shown below −

```
{
    _id: ,
    Emp_ID: "10025AE336"
    Personal_details:{
        First_Name: "Radhika",
        Last_Name: "Sharma",
        Date_Of_Birth: "1995-09-26"
    },
    Contact: {
        e-mail: "radhika_sharma.123@gmail.com",
        phone: "9848022338"
    },
    Address: {
        city: "Hyderabad",
        Area: "Madapur",
        State: "Telangana"
    }
}
```

Data Model conti.

Normalized Data Model

In this model, you can refer the sub documents in the original document, using references. For example, you can re-write the above document in the normalized model as:

Employee:

```
{
     _id: <ObjectId101>,
     Emp_ID: "10025AE336"
}
```

Personal_details:

```
{
     _id: <ObjectId102>,
     empDocID: " ObjectId101",
     First_Name: "Radhika",
     Last_Name: "Sharma",
     Date_Of_Birth: "1995-09-26"
}
```

Contact:

```
{
     _id: <ObjectId103>,
     empDocID: " ObjectId101",
     e-mail: "radhika_sharma.123@gmail.com",
     phone: "9848022338"
}
```

Address:

```
{
     _id: <ObjectId104>,
     empDocID: " ObjectId101",
     city: "Hyderabad",
     Area: "Madapur",
     State: "Telangana"
}
```

Parul® University

Data Model conti.

Considerations while designing Schema in MongoDB

- Design your schema according to user requirements.
- Combine objects into one document if you will use them together. Otherwise separate them (but make sure there should not be need of joins).
- Duplicate the data (but limited) because disk space is cheap as compare to compute time.
- Do joins while write, not on read.
- Optimize your schema for most frequent use cases.
- Do complex aggregation in the schema.

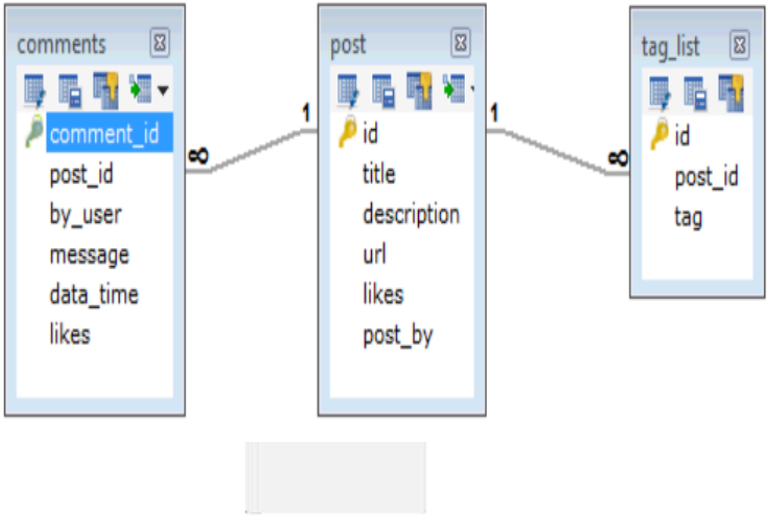
Example

- Suppose a client needs a database design for his blog/website and see the differences between RDBMS and MongoDB schema design. Website has the following requirements.
- Every post has the unique title, description and url.
- Every post can have one or more tags.
- Every post has the name of its publisher and total number of likes.
- Every post has comments given by users along with their name, message, data-time and likes. On each post, there can be zero or more comments.

Parul® University

Data Model conti.

In RDBMS schema, design for above requirements will have minimum three tables.




```
{
   _id: POST_ID
   title: TITLE_OF_POST,
   description: POST_DESCRIPTION,
   by: POST_BY,
   url: URL_OF_POST,
   tags: [TAG1, TAG2, TAG3],
   likes: TOTAL_LIKES,
   comments: [
      {
         user:'COMMENT_BY',
         message: TEXT,
         dateCreated: DATE_TIME,
         like: LIKES
      },
      {
         user:'COMMENT_BY',
         message: TEXT,
         dateCreated: DATE_TIME,
         like: LIKES
      }
   ]
}
```

While in MongoDB schema, design will have one collection post

Parul® University

Commands (Basic commands)

Category	Command & Example	Description
Connect	mongosh "mongodb://localhost:27017"	Opens MongoDB shell to local server (medium.com)
Show Databases	show dbs	Lists all databases
Switch/Create DB	use myDB	Switches to or creates myDB
Current DB	db	Displays current database context
Show Collections	show collections	Lists collections in current DB
Create Collection	db.createCollection("myCol")	Explicitly creates myCol collection
Drop Collection	db.myCol.drop()	Deletes myCol
Database Stats	db.stats()	Returns database usage stats
Count Documents	db.users.count()	Shows number of documents in users
Clear Shell	cls or Ctrl + L	Clears the console screen

Parul® University

Commands

CRUD Operations

Action	Command & Example	Description
Insert One	db.users.insertOne({name:"Alice", age:25})	Inserts a single document into users (geeksforgeeks.org, tutorialsteacher.com)
Insert Many	db.users.insertMany([{name:"Bob"},{name:"Carol"}])	Inserts multiple documents
Find All	db.users.find()	Retrieves all documents
Find One	db.users.findOne({age:{$gt:30}})	Fetches first document where age > 30
Limit/Sort	db.users.find().limit(5).sort({age:-1})	Fetches first 5 sorted by descending age
Update One	db.users.updateOne({name:"Alice"}, {$set:{age:26}})	Updates one matching document
Update Many	db.users.updateMany({age:{$lt:20}}, {$inc:{age:1}})	Increments age by 1 for all matching
Delete One	db.users.deleteOne({name:"Bob"})	Deletes one matching document
Delete Many	db.users.deleteMany({age:{$gte:60}})	Deletes all where age ⩾ 60
Find & Modify	db.users.findOneAndUpdate({name:"Carol"}, {$set:{age:30}})	Returns the updated document
Aggregate	db.orders.aggregate([{$match:{status:"shipped"}}, {$group:{_id: "$custId", total:{$sum:"$price"}}}])	Performs grouping and summation

Parul® University

Additional Useful Commands

Command	Purpose
db.help(), db.col.help()	Show shell and collection methods help (blog.e-zest.com, mongodb-devhub-cms.s3.us-west-1.amazonaws.com)
db.stats()	Display current database stats
db.col.count()	Count documents in a collection
db.col.aggregate([...])	Run aggregation pipelines
exit or Ctrl+C Ctrl+C	Exit the mongosh shell
cls or Ctrl+L	Clear the shell console

Parul® University

Mongo DB Operators

MongoDB Query Operators

There are many query operators that can be used to compare and reference document fields.

Comparison

The following operators can be used in queries to compare values:

- $eq: Values are equal
- $ne: Values are not equal
- $gt: Value is greater than another value
- $gte: Value is greater than or equal to another value
- $lt: Value is less than another value
- $lte: Value is less than or equal to another value
- $in: Value is matched within an array

Parul® University

Example (Comparison Operator)

Operator	Example	Description
$eq	db.col.find({ age: { $eq: 30 } })	Matches field exactly equal to a value (mongodb.com, youtube.com)
$ne	db.col.find({ status: { $ne: "inactive" } })	Matches field not equal to a value
$gt	db.inv.find({ qty: { $gt: 100 } })	Finds values greater than a threshold
$lt	db.inv.find({ qty: { $lt: 50 } })	Finds values less than a threshold
$gte, $lte	db.col.find({ age: { $gte: 18, $lte: 65 } })	Reads within an inclusive range
$in	db.col.find({ country: { $in: ["US","UK"] } })	Matches any value in a list
$nin	db.col.find({ country: { $nin: ["US","UK"] } })	Excludes values in a list

Parul® University

Mongo DB Operators

Logical

The following operators can logically compare multiple queries.

- $and: Returns documents where both queries match
- $or: Returns documents where either query matches
- $nor: Returns documents where both queries fail to match
- $not: Returns documents where the query does not match

Evaluation

The following operators assist in evaluating documents.

- $regex: Allows the use of regular expressions when evaluating field values
- $text: Performs a text search
- $where: Uses a JavaScript expression to match documents

Parul® University

Mongo DB Operators

MongoDB Update Operators

There are many update operators that can be used during document updates.

Fields

The following operators can be used to update fields:

$currentDate: Sets the field value to the current date

$inc: Increments the field value

$rename: Renames the field

$set: Sets the value of a field

$unset: Removes the field from the document

Array

The following operators assist with updating arrays.

$addToSet: Adds distinct elements to an array

$pop: Removes the first or last element of an array

$pull: Removes all elements from an array that match the query

$push: Adds an element to an array

Parul® University

Mongo CRUD Operation

Operation	Command & Example	Description
Create	db.users.insertOne({ name: "Alice", age: 30 }) db.users.insertMany([{ name: "Bob" }, { name: "Carol" }]) (geeksforgeeks.org, geeksforgeeks.org)	Inserts single or multiple documents into the users collection.
Read	db.users.find() db.users.find({ age: { $gt: 25 } }) db.users.findOne({ name: "Alice" })	Retrieves documents; supports filters and returns either cursor or single document.
Update	db.users.updateOne({ name: "Alice" }, { $set: { age: 31 } }) db.users. updateMany({ age: { $lt: 25 } }, { $inc: { age: 1 } })	Modifies one or more documents using update operators like $set, $inc.
Replace	db.users.replaceOne({ name: "Bob" }, { name: "Bob", age: 28 })	Replaces a document entirely, rather than updating specific fields.
Delete	db.users.deleteOne({ name: "Alice" }) db.users.deleteMany({ age: { $gt: 30 } })	Removes single or multiple documents from the collection based on filter criteria.
Find + Modify	db.users.findOneAndUpdate({ name: "Carol" }, { $set: { city: "Mumbai" } })	Performs an update and returns the updated document in one atomic operation.
Bulk Write	Use bulk operations like bulkWrite() for batching multiple CRUD actions.	Enables efficient execution of multiple write operations in a single batch (e.g., inserts, updates, deletes).

DIGITAL LEARNING CONTENT

Parul® University

www.paruluniversity.ac.